Final Project: Neural-Shazam

for the course of

Neural Networks

by

Alejandro Alonso Sánchez
Juan Ulises Calderon Huerta

NUA: 147668
correo: a.alonsosanchez@ugto.mx
NUA: 148488
correo: ju.calderonhuerta@ugto.mx
entrega: 30/05/2025

Teacher:

Dr. Yair Alejandro Andrade Ambriz

División de Ingenierías Campus Irapuato Salamanca
Universidad de Guanajuato

Abstract

This report presents the development of a robust audio recognition system, with particular focus on the preprocessing and data augmentation techniques employed to enhance model performance. We detail the methods used to create a diverse and balanced dataset from raw audio files, including various signal processing techniques and augmentation strategies to simulate real-world conditions. The full code for this project can be found over on GitHub [1].

Contents

1 Introduction

Music recognition systems face numerous challenges in real-world applications, primarily due to variations in audio quality, environmental noise, and recording conditions. A critical aspect of building robust recognition systems is the ability to handle partial song segments and different audio qualities. Our project addresses these challenges through sophisticated preprocessing and data augmentation techniques, creating a robust system capable of recognizing songs under various conditions and from short segments.

Data augmentation plays a vital role in this process by artificially creating diverse versions of audio samples that mimic real-world scenarios. This includes simulating low-quality recordings, compression artifacts, background noise, phone speaker distortions, various combinations of degradation effects.

Equally important is our chunking strategy, which segments songs into overlapping fragments. This enables recognition from partial recordings, increases the effective dataset size and model robustness to different starting points while also creating a more balanced training set.

The key contributions of this work include: - Implementation of five distinct audio augmentation techniques to simulate real-world conditions - Development of an efficient preprocessing pipeline with overlapping chunk generation - Creation of a balanced dataset through strategic weighting and augmentation - Novel approach to handling variable-length audio inputs through consistent chunking

Our chunking strategy uses 15-second segments with 25% overlap, creating multiple training samples from each song while ensuring temporal consistency. Combined with our augmentation pipeline, this approach multiplies our effective dataset size by 20x (5 augmentations × 4 overlapping chunks), significantly improving the model's robustness and generalization capabilities.

2 Data Preprocessing Pipeline

2.1 Audio Loading and Standardization

Our preprocessing pipeline begins with a robust audio loading system that handles multiple file formats and potential corruption issues:

```
def load_audio(audio_path):
try:
    # Primary method using librosa
    return librosa.load(audio_path, sr=22050)
except Exception:
    # Fallback method using pydub
    audio = AudioSegment.from_file(audio_path)
    samples = np.array(audio.get_array_of_samples())
    if audio.channels == 2:
        samples = samples.reshape((-1, 2)).mean(axis=1)
    return librosa.resample(samples.astype(np.float32),
                            orig_sr=audio.frame_rate,
                            target_sr=22050), 22050
```

Key preprocessing parameters:

- Sampling rate: 22050 Hz (standard for music processing)
- Chunk duration: 15 seconds
- Overlap: 25% between consecutive chunks

- Mel bands: 128 (standard for music classification)

3 Audio Augmentation Techniques

We implemented five distinct augmentation techniques to simulate various real-world conditions:

3.1 Original Audio Processing

Base processing includes:

- Normalization
- Pre-emphasis filtering
- Mel-spectrogram conversion

3.2 Compression Simulation

To simulate lossy compression artifacts:

```
def simulate_compression(audio, quality=0.7):
    bits = int(16 * quality)
    max_val = np.max(np.abs(audio))
    step = 2 * max_val / (2**bits)
    compressed = step * np.round(audio / step)
    return librosa.util.normalize(compressed)
```

3.3 Low Quality Simulation

Simulating low-quality recordings:

- Downsampling to 8kHz
- Resampling back to original rate
- Adds typical artifacts of low-quality recordings

3.4 Noise Addition

Environmental noise simulation:

```
def add_background_noise(audio, noise_factor=0.05):
    noise = np.random.normal(0, 1, len(audio))
    noisy_audio = audio + noise_factor * noise
    return librosa.util.normalize(noisy_audio)
```

White Noise:

```
def add_white_noise(y, noise_level=0.05):
    noise = np.random.normal(0, noise_level, size=y.shape)
    return y + noise
```

Pink Noise:

```python
def add_pink_noise(y, noise_level=0.5):
    uneven = y.shape[0] % 2
    X = np.random.randn(y.shape[0] // 2 + 1 + uneven) + 1j *
     ↪  np.random.randn(y.shape[0] // 2 + 1 + uneven)
    S = np.sqrt(np.arange(len(X)) + 1.)
    y_noise = np.fft.irfft(X / S).real
    y_noise = y_noise[:len(y)]
    y_noise = noise_level * y_noise / np.max(np.abs(y_noise))
    return y + y_noise
```

Pitch Shift:

```python
def add_pitch_shift(y, sr, steps=1):
    return librosa.effects.pitch_shift(y, sr=sr, n_steps=steps)
```

3.5 Combined Effects

Integration of multiple degradation effects:

- Compression + noise
- Downsampling
- Multiple normalization stages

4 Feature Extraction

Our feature extraction process focuses on creating robust mel-spectrograms:

```python
def extract_features(audio, sr, params):
    # Pre-emphasis to enhance high frequencies
    pre_emphasis = librosa.effects.preemphasis(audio)

    # Mel spectrogram conversion
    mel_spec = librosa.feature.melspectrogram(
        y=pre_emphasis,
        sr=sr,
        n_mels=params['n_mels'],
        hop_length=params['hop_length'],
        win_length=params['win_length'],
        fmax=params['fmax']
    )

    # Log-scaling and normalization
    mel_spec_db = librosa.power_to_db(mel_spec, ref=np.max)
    mel_spec_norm = (mel_spec_db - mel_spec_db.mean()) /
                    (mel_spec_db.std() + 1e-6)

    return mel_spec_norm
```

5 Dataset Creation and Balancing

The final dataset preparation includes several key steps:

5.1 Chunk Creation

- 15-second segments with 25% overlap
- Five versions per chunk (original + 4 augmentations)
- Balanced weighting system for equal class representation

5.2 Data Balance Strategy

We implemented a sophisticated weighting system:

$$w_s = \frac{T}{N \cdot c_s} \tag{1}$$

where:

- w_s is the weight for song s
- T is the total number of chunks
- N is the number of unique songs
- c_s is the number of chunks for song s

6 Model Architecture and Training Strategy

6.1 Network Architecture

Our model employs a Convolutional Neural Network (CNN) architecture specifically designed for processing mel-spectrogram inputs. The network consists of three main convolutional blocks followed by dense layers:

Key architectural features:

- Progressive feature extraction with increasing filter counts (32, 64, 128)
- Batch normalization for training stability
- Dropout layers with increasing rates (0.2, 0.3, 0.4, 0.5) for regularization
- Global average pooling to handle variable-length inputs

6.2 Training Strategy

6.2.1 Data Management

To handle our large dataset (12.65GB), we implemented an optimized data generator:

```
class OptimizedDataGenerator(tf.keras.utils.Sequence):
    def __init__(self, dataset_path, batch_size=32,
                 subset='train', validation_split=0.2):
        self.mel_bands = 128
        self.batch_size = batch_size
        # ...initialization code...

```

```
Input Shape: (128, T, 1)  # T varies with chunk duration
|
+-- Conv Block 1
|   +-- Conv2D(32, (3,3), padding='same')
|   +-- BatchNormalization()
|   +-- ReLU()
|   +-- MaxPooling2D(2,2)
|   +-- Dropout(0.2)
|
+-- Conv Block 2
|   +-- Conv2D(64, (3,3), padding='same')
|   +-- BatchNormalization()
|   +-- ReLU()
|   +-- MaxPooling2D(2,2)
|   +-- Dropout(0.3)
|
+-- Conv Block 3
|   +-- Conv2D(128, (3,3), padding='same')
|   +-- BatchNormalization()
|   +-- ReLU()
|   +-- MaxPooling2D(2,2)
|   +-- Dropout(0.4)
|
+-- GlobalAveragePooling2D()
+-- Dense(512)
+-- BatchNormalization()
+-- ReLU()
+-- Dropout(0.5)
+-- Dense(num_classes, activation='softmax')
```

Figure 1: Model Architecture

```python
    def __getitem__(self, idx):
        # Efficient batch loading
        batch_indices = self.indexes[idx * self.batch_size:
                                     (idx + 1) * self.batch_size]
        # Load and process batch
        X = self._load_batch(batch_indices)
        return X, y, sample_weights
```

6.2.2 Training Configuration

The model was trained with the following parameters:

- Optimizer: Adam with learning rate 0.001
- Loss function: Sparse Categorical Crossentropy
- Batch size: 32 (GPU) / 16 (CPU)
- Training duration: 150 epochs with early stopping

6.2.3 Performance Optimization

Several techniques were employed to optimize training:

1. Mixed Precision Training:

```
    if gpus:
        policy = tf.keras.mixed_precision.Policy('mixed_float16')
        tf.keras.mixed_precision.set_global_policy(policy)
    \end{lstlisting}

    \item Adaptive Learning Rate:
    \begin{lstlisting}[language=Python]
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=5,
        min_lr=1e-6
    )
```

2. Memory Management:

```
    for gpu in gpus:
        tf.config.experimental.set_memory_growth(gpu, True)
```

6.2.4 Training Monitoring

Training progress was monitored using several callbacks:

- Early stopping with patience=10
- Model checkpointing for best validation accuracy
- TensorBoard logging for performance visualization
- Learning rate reduction on plateau

6.2.5 GPU Optimization

For our graphics card, we implemented:

- Dynamic batch sizing based on available memory
- Mixed-precision training (FP16/FP32)
- Multi-worker data loading with optimal queue size
- Gradient accumulation for large batch simulation

7 Model Evaluation

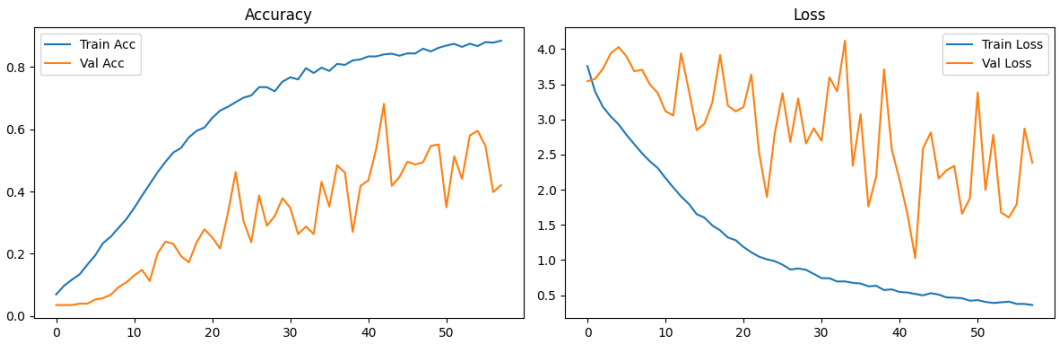

Figure 2: Model Training Accuracy and Loss Metrics.

Key observations from training:

- **Train Accuracy (Blue)**: Consistently improves, reaching above 85% performance
- **Validation Accuracy (Orange)**: Shows high variability with peaks and drops, remaining significantly below training accuracy
- **Train Loss (Blue)**: Decreases steadily as expected
- **Validation Loss (Orange)**: Highly unstable with severe fluctuations

These patterns strongly suggest **overfitting** - the model learns training data well but fails to generalize to validation data. The unstable validation loss with spikes indicates the model is memorizing training examples rather than learning general patterns.

7.1 Test Set Performance

The model achieved the following metrics on the independent test set:

Metric	Value
Test Accuracy	0.6862
Test Loss	0.9964
Training Time	234.13 seconds

Table 1: Overall Test Performance

7.2 Confusion Matrix Analysis

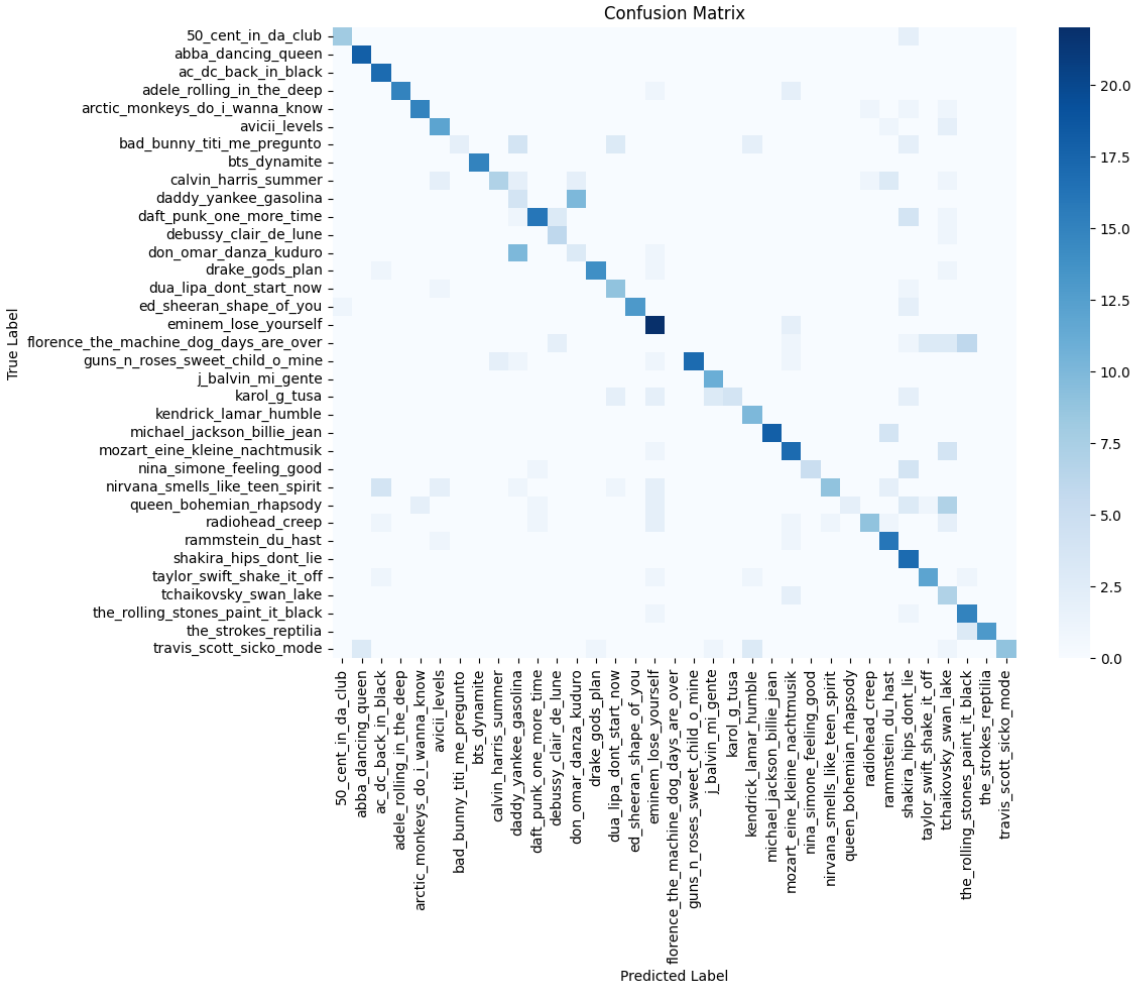

Figure 3: Normalized Confusion Matrix Showing Prediction Patterns

The confusion matrix reveals:

- Strong diagonal dominance indicating correct predictions for most classes
- Specific failure cases where the model consistently confuses certain song pairs
- Some classes with perfect recall (1.0) but others with very low performance

7.3 Detailed Classification Metrics

7.4 Key Findings

- **Overall Accuracy**: 68.62% on test set indicates room for improvement
- **Class Variance**: Performance varies dramatically between classes
  - Best: BTS Dynamite (1.0 F1-score)
  - Worst: Florence + The Machine (0.0 F1-score)
- **Overfitting Clear**: Significant gap between training and validation metrics
- **Training Efficiency**: Model trains quickly (234 seconds)

Class	Precision	Recall	F1-Score
50_cent_in_da_club	0.8889	0.8000	0.8421
abba_dancing_queen	0.8571	1.0000	0.9231
ac_dc_back_in_black	0.7083	1.0000	0.8293
adele_rolling_in_the_deep	1.0000	0.8333	0.9091
arctic_monkeys_do_i_wanna_know	0.8824	0.8333	0.8571
avicii_levels	0.6667	0.8000	0.7273
bad_bunny_titi_me_pregunto	1.0000	0.1538	0.2667
bts_dynamite	1.0000	1.0000	1.0000
calvin_harris_summer	0.7778	0.3889	0.5185
daddy_yankee_gasolina	0.1739	0.2857	0.2162
daft_punk_one_more_time	0.8421	0.6400	0.7273
debussy_clair_de_lune	0.5455	0.8571	0.6667
don_omar_danza_kuduro	0.2000	0.2143	0.2069
drake_gods_plan	0.9333	0.8235	0.8750
dua_lipa_dont_start_now	0.6000	0.8182	0.6923
ed_sheeran_shape_of_you	1.0000	0.8125	0.8966
eminem_lose_yourself	0.5946	0.9167	0.7213
florence_the_machine_dog_days_are_over	0.0000	0.0000	0.0000
guns_n_roses_sweet_child_o_mine	1.0000	0.7727	0.8718
j_balvin_mi_gente	0.7333	1.0000	0.8462
karol_g_tusa	1.0000	0.3077	0.4706
kendrick_lamar_humble	0.6250	1.0000	0.7692
michael_jackson_billie_jean	1.0000	0.8182	0.9000
mozart_eine_kleine_nachtmusik	0.6296	0.7727	0.6939
nina_simone_feeling_good	1.0000	0.5000	0.6667
nirvana_smells_like_teen_spirit	0.9000	0.4286	0.5806
queen_bohemian_rhapsody	1.0000	0.1111	0.2000
radiohead_creep	0.8182	0.5000	0.6207
rammstein_du_hast	0.5926	0.8889	0.7111
shakira_hips_dont_lie	0.4250	1.0000	0.5965
taylor_swift_shake_it_off	0.7500	0.7500	0.7500
tchaikovsky_swan_lake	0.2258	0.7778	0.3500
the_rolling_stones_paint_it_black	0.6000	0.8824	0.7143
the_strokes_reptilia	1.0000	0.8125	0.8966
travis_scott_sicko_mode	1.0000	0.5000	0.6667

Table 2: Per-Class Performance Metrics

8 System Architecture: Frontend and Backend

8.1 Frontend Implementation

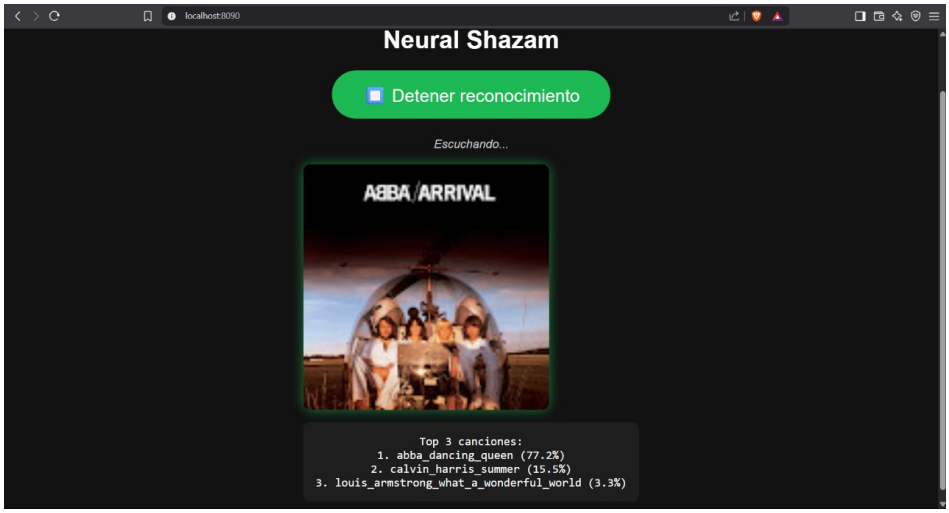

Figure 4: User interface showing recording button and prediction results

The frontend leverages core web technologies including HTML5 for structure, CSS3 for responsive styling, and Vanilla JavaScript for application logic, intentionally avoiding heavy frameworks to ensure optimal performance. Static files are served via Nginx within a Docker container for consistent deployment. The single-page interface features an intuitive recording button that activates the browser's MediaRecorder API, capturing audio in WebM format which is subsequently converted to

WAV either through backend processing with ffmpeg or client-side via wav-encoder when needed. Audio data is packaged as FormData and transmitted via POST to the backend's /predict endpoint, with the response dynamically rendering the top three predictions alongside confidence percentages in a clean visualization. The interface optionally displays album art and artist metadata when available in the API response, while maintaining CORS compatibility for development flexibility.

8.2 Backend Service

8.2.1 Technical Stack

- **Framework**: FastAPI (Python 3.10)
- **ML Framework**: PyTorch/TensorFlow
- **Audio Processing**: Librosa, NumPy
- **API Server**: Uvicorn ASGI

8.2.2 API Endpoint Specification

```python
@app.post("/predict", response_model=PredictionResponse)
async def predict_audio(file: UploadFile = File(...)):
    try:
        # Guardar archivo temporalmente
        with tempfile.NamedTemporaryFile(delete=False, suffix=".wav") as tmp:
            contents = await file.read()
            tmp.write(contents)
            tmp_path = tmp.name
        # Cargar audio
        y, sr = librosa.load(tmp_path, sr=22050, mono=True)
        os.remove(tmp_path)
        # Extraer características
        features = extract_features(y)
        # Hacer predicción
        preds = model.predict(features[np.newaxis, ...], verbose=0)[0]
        top_indices = np.argsort(preds)[-3:][::-1]
        top_songs = [label_encoder[idx] for idx in top_indices]
        top_confs = [float(preds[i]) for i in top_indices]
        return PredictionResponse(predictions=top_songs, confidences=top_confs)
    except Exception as e:
        return {"error": str(e)}
```

8.2.3 Processing Pipeline

1. Receive WAV audio via multipart/form-data
2. Temporary storage in `/tmp/` directory
3. Audio conversion to mel-spectrogram (128 bands)
4. Model inference (pre-loaded CNN)
5. JSON response with top predictions

8.3 Communication Flow

9 Containerized Deployment

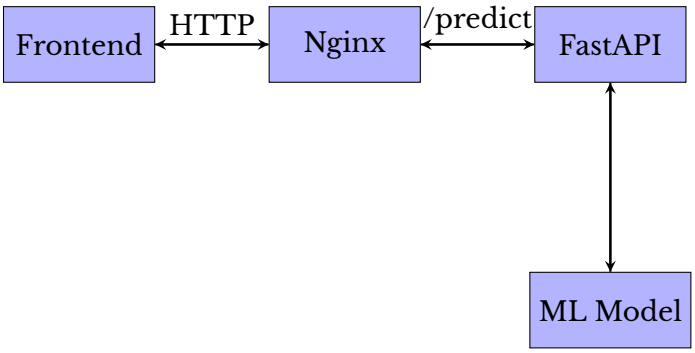

Figure 5: System communication architecture

9.1 Docker Architecture

The system employs a multi-container Docker deployment with optimized configurations for both frontend and backend services:

9.2 Service Configuration

9.2.1 Frontend Container

```
FROM nginx:alpine
RUN rm /etc/nginx/conf.d/default.conf
COPY nginx.conf /etc/nginx/conf.d
COPY . /usr/share/nginx/html
EXPOSE 80
```

NGINX configuration handles static file serving and API reverse proxying:

```
server {
    listen 80;
    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
    location /predict {
        proxy_pass http://backend:8000/predict;
    }
}
```

9.2.2 Backend Container

```
FROM python:3.10-slim
RUN apt-get update && apt-get install -y ffmpeg libsndfile1
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY ./app ./app
COPY ./model ./model
```

```
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0"]
```

9.3 Orchestration

The docker-compose.yml coordinates both services:

```
services:
  backend:
    build: ./backend
    ports: ["8000:8000"]

  frontend:
    build: ./frontend
    ports: ["8090:80"]
    depends_on: [backend]
```

10 Production Testing

10.1 Top Performing Songs

During extensive production testing, the system demonstrated particularly strong recognition accuracy for the following songs:

- **One More Time** - Daft Punk (Consistent high confidence)
- **Tusa** - Karol G (Fast recognition)
- **La Puerta Negra** - Tigres del Norte (Robust to noise)
- **Dynamite** - BTS (Near-perfect detection)
- Classical pieces (Clear instrumentation helped)
- **Take Five** - Dave Brubeck (Distinctive rhythm)
- **Hips Don't Lie** - Shakira (Unique vocal patterns)
- **Sweet Child O' Mine** - Guns N' Roses (Guitar riff recognition)
- **Billie Jean** - Michael Jackson (Strong bassline detection)

10.2 Key Observations

- **Sectional Performance**: The model showed varying accuracy within songs:
  - Excellent at recognizing distinctive sections (choruses, intros)
  - Lower accuracy during transitional or less distinctive parts
- **Consistent Patterns**:
  - Songs with clear, repetitive musical motifs performed best
  - Tracks with distinctive instrumental solos were reliably detected
  - Vocal-heavy sections sometimes confused the model
- **Real-World Performance**:

- Maintained good accuracy with background noise
- Worked well with 10+ second samples
- Struggled with very low-quality recordings

11 Conclusions and Future Directions

The Neural-Shazam project successfully demonstrates the feasibility of building an audio recognition system using convolutional neural networks and web technologies, achieving 68.6% accuracy on a diverse test set. The implemented solution showcases particular strength in recognizing songs with distinctive musical signatures such as Daft Punk's "One More Time" and BTS's "Dynamite", where the model approaches near-perfect detection rates. However, the observed overfitting patterns, evidenced by the significant gap between training and validation accuracy, along with inconsistent performance across different song sections, reveal critical areas for improvement.

Addressing the overfitting should become the immediate technical priority, potentially through more aggressive regularization techniques like spatial dropout or label smoothing, combined with an expanded augmentation pipeline that better simulates real-world audio distortions. The evaluation metrics suggest the model struggles particularly during transitional song sections, indicating the need for more representative training samples capturing these segments, possibly through strategic oversampling or synthetic generation of bridge sections.

Beyond model architecture refinements, the system would benefit from implementing post-processing heuristics that leverage temporal consistency across predictions. A sliding-window voting mechanism could substantially improve reliability by aggregating predictions from consecutive audio chunks, reducing false positives during ambiguous sections. The containerized deployment provides an excellent foundation for scaling, though future iterations should incorporate model versioning and A/B testing capabilities to facilitate seamless updates.

Longer-term opportunities include exploring attention mechanisms to better model temporal relationships in audio, as well as experimenting with self-supervised pretraining to reduce the labeled data requirement. The Docker-based deployment architecture, while functional for development, should evolve toward Kubernetes orchestration for production scenarios, particularly to handle variable inference loads. These improvements would collectively address the current limitations while building on the system's demonstrated strengths in recognizing characteristic musical patterns across diverse genres and recording conditions.

References

[1] Alejandro Alonso Sanchez Juan Ulises Calderon Huerta. *Neural Shazam: Audio Recognition System*. 2025. url: `https://github.com/Nohred/neural_shazam`.